

Object-Oriented Programming (OOP) in Python

Technical Notes

Rodrigo Kang

Table of contents

1	Object-Oriented Programming	4
2	Classes and Objects	6
3	Attributes	7
3.1	Instance Attributes	8
3.2	Class Attributes	9
4	Object Initialisation	10
5	The self Parameter	11
6	Default Attribute Values	13
7	Inspecting Instance Attributes	14
8	Methods	15
8.1	Methods that Modify State	16
9	Instance Methods	17
10	Methods with Additional Parameters	19
11	Returning Values from Methods	20
12	Classes as Abstractions	21
13	OOP in Data Science and Machine Learning	22
14	Functions or Classes?	23
15	Static Methods	24

16 Class Methods	27
16.1 Alternative Constructors	28
16.2 Instance, Static, and Class Methods	29
17 Properties	30
17.1 Computed Attributes	32
18 Controlled Attribute Access	33
19 Public and Non-Public Attributes	35
20 Encapsulation	36
21 Special Methods	38
22 String Representation with <code>__str__()</code>	39
23 <code>__repr__()</code> and <code>__str__()</code>	41
24 Equality and Object Identity	43
25 Defining Equality with <code>__eq__()</code>	44
26 Other Special Methods	46
27 A Note on <code>__del__()</code>	47
28 Designing a Small Class	48
29 OOP and Interface Design	51
30 Inheritance	51
31 Single Inheritance	54
32 Inheritance Chains	56
33 Extending a Parent Class	57
34 <code>super()</code>	58
35 Method Overriding	59
36 Extending an Overridden Method	60
37 Polymorphism	62
37.1 Polymorphism through Methods	63
38 Duck Typing	64
39 Polymorphism in Machine Learning APIs	65

40 Multiple Inheritance	66
41 Method Resolution Order	68
42 super() and the MRO	69
43 Cooperative Multiple Inheritance	71
44 Inheritance from Built-In Classes	72
45 When Inheritance Is Appropriate	73
46 Composition	74
46.1 Composition versus Inheritance	75
47 Composition in Machine Learning Systems	76
48 Prefer Composition or Inheritance?	77
49 Dataclasses	77
50 Default Values in Dataclasses	79
51 Dataclasses Are Still Classes	79
52 When to Use a Dataclass	80
53 OOP as a Design Choice	81
54 OOP in a Machine Learning Workflow	82
55 Learned Attributes	86
56 Interface Consistency	86
57 Inheritance versus Interface	87
58 Problems	88
58.1 Problem 1 — RationalNumber	88
58.2 Problem 2 — String Representation	89
58.3 Problem 3 — Rational Number Behaviour	90
58.4 Problem 4 — Static Methods for Rational Arithmetic	91
58.5 Problem 5 — Alternative Constructors	91
58.6 Problem 6 — Properties in RationalNumber	92
58.7 Problem 7 — Point3D from Point2D	93
58.8 Problem 8 — Multiple Inheritance and Pyramid	94
58.9 Problem 9 — Student	95
58.10 Problem 10 — Methods with Additional Arguments	96
58.11 Problem 11 — Instance Independence	97
58.12 Problem 12 — Counting Instances	97

58.13	Problem 13 — Sensor	98
58.14	Problem 14 — Overriding with Accelerometer	99
58.15	Problem 15 — Extending a Parent Initialiser	100
58.16	Problem 16 — RegularPolygon	101
58.17	Problem 17 — Polygon Subclasses	102
58.18	Problem 18 — Three-Dimensional Shapes	102
58.19	Problem 19 — Circle	103
58.20	Problem 20 — Cylinder: Inheritance or Composition?	104
58.21	Problem 21 — Identity Versus Equality	105
58.22	Problem 22 — A Read-Only Property	105
58.23	Problem 23 — Class, Static, or Instance Method?	106
58.24	Problem 24 — Duck Typing	106
58.25	Problem 25 — Polymorphic Interface	107
58.26	Problem 26 — Method Resolution Order	108
58.27	Problem 27 — Cooperative super()	109
58.28	Problem 28 — Composition	109
58.29	Problem 29 — Dataclass	110
58.30	Problem 30 — Mutable Defaults in Dataclasses	110
58.31	Problem 31 — Dataset Abstraction	111
58.32	Problem 32 — Stateful Standardizer	112
58.33	Problem 33 — Separate Fit and Transform Data	112
58.34	Problem 34 — Alternative Constructor for a Model Configuration	113
58.35	Problem 35 — Inheritance or Composition?	113
58.36	Problem 36 — Refactoring a Procedural Workflow	114
58.37	Problem 37 — Two Independently Fitted Objects	115
58.38	Problem 38 — Interface-Based Evaluation	115
58.39	Problem 39 — Diagnose a Poor Class	116
58.40	Problem 40 — Integrated OOP Reasoning	116

1 Object-Oriented Programming

```

1 import math
2 import numpy as np

```

Object-oriented programming (OOP) is a programming paradigm in which data and the operations associated with that data are organised into **objects**.

An object combines two broad components:

- **state**, represented by data stored in attributes;
- **behaviour**, represented by methods that operate on that state.

Rather than treating data and functions as completely separate entities, object-oriented programming allows related data and behaviour to be represented together.

In Python, OOP is particularly important because the language itself is strongly object-oriented. Integers, strings, lists, functions, DataFrames, fitted machine learning models, and many other structures encountered in scientific computing are objects.

For example:

```
1 x = 10
2 text = "Python"
3 values = [1, 2, 3]
4
5 print(type(x))
6 print(type(text))
7 print(type(values))
```

```
<class 'int'>
<class 'str'>
<class 'list'>
```

Each object belongs to a particular **class**.

The class determines the general structure and behaviour available to objects of that type.

For example, a list provides methods such as:

```
1 values.append(4)
2
3 values
```

```
[1, 2, 3, 4]
```

The method `.append()` is behaviour defined by the list class and made available to list objects.

Object-oriented programming therefore provides a way to define new types of objects with their own:

- attributes;
- methods;
- validation rules;
- relationships with other objects.

2 Classes and Objects

A **class** defines the structure and behaviour associated with a type of object.

An **object**, also called an **instance**, is a concrete object created from that class.

Conceptually:

class $\rightarrow$ instances.

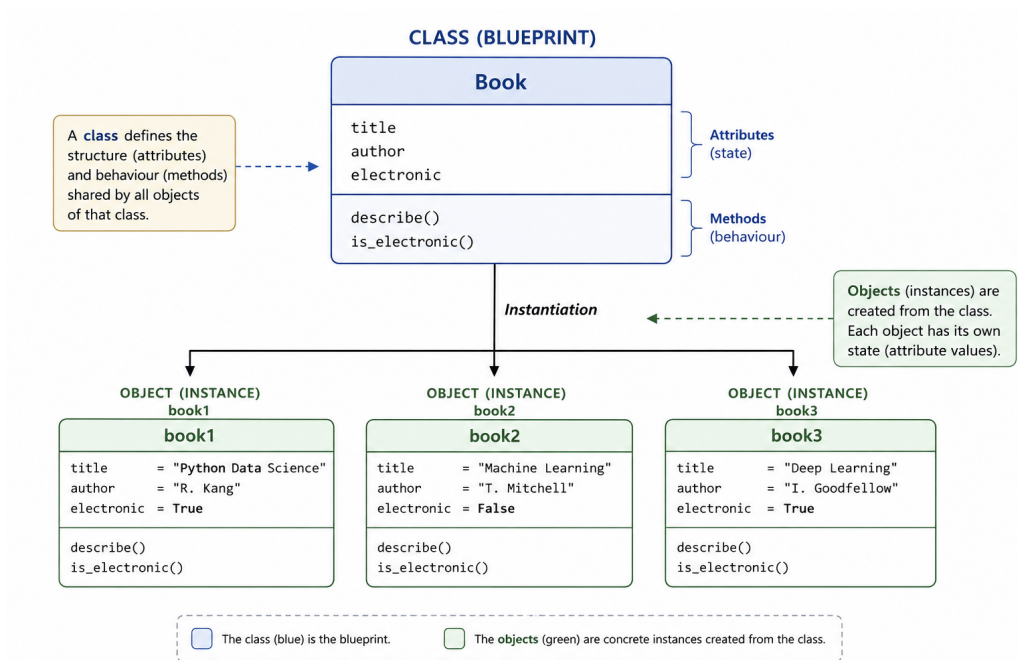


Figure 1: A class defines a common structure and behaviour from which multiple independent objects can be instantiated. Each object is an instance of the class and maintains its own state.

For example, suppose that an application needs to represent books.

A class named `Book` can define what information and behaviour should be associated with a book, while individual books are represented by instances of that class.

A minimal class can be defined using the class statement:

```
1 class Book:
2     pass
```

The class now exists as a new Python type:

```
1 Book
```

```
__main__.Book
```

An instance can be created by calling the class:

```
1 book = Book()
```

Its type is:

```
1 type(book)
```

```
__main__.Book
```

The relationship is:

```
1 isinstance(book, Book)
```

```
True
```

Thus:

```
1 Book
2   class
3
4 book
5   instance of Book
```

A class should not be thought of merely as a container for variables and functions. It defines an abstraction representing a particular kind of object in the program.

3 Attributes

Objects usually contain data describing their current state.

These values are stored in **attributes**.

There are two important kinds of attributes to distinguish:

- instance attributes;
- class attributes.

3.1 Instance Attributes

An instance attribute belongs to a particular object.

For example, different books may have different titles and authors.

Instance attributes are commonly created during object initialisation.

```
1 class Book:
2
3     def __init__(self, title, author):
4         self.title = title
5         self.author = author
```

Two instances can now be created:

```
1 book1 = Book(
2     title="The Lord of the Rings",
3     author="J.R.R. Tolkien"
4 )
5
6 book2 = Book(
7     title="The Hobbit",
8     author="J.R.R. Tolkien"
9 )
```

Their attributes can be accessed using dot notation:

```
1 book1.title
```

'The Lord of the Rings'

```
1 book2.title
```

'The Hobbit'

Although both objects belong to the same class, each instance maintains its own state.

Conceptually:

```

1 Book
2   |— book1
3   |   |— title = "The Lord of the Rings"
4   |   |— author = "J.R.R. Tolkien"
5   |
6   |— book2
7   |   |— title = "The Hobbit"
8   |   |— author = "J.R.R. Tolkien"

```

This distinction between the **definition of a type** and the **state of individual instances** is fundamental to object-oriented programming.

3.2 Class Attributes

A class attribute belongs to the class itself and is shared conceptually by its instances unless shadowed at the instance level.

For example:

```

1 class Book:
2
3     category = "book"
4
5     def __init__(self, title, author):
6         self.title = title
7         self.author = author

```

The class attribute can be accessed through the class:

```

1 Book.category

```

'book'

and through an instance:

```

1 book = Book(
2     title="The Hobbit",
3     author="J.R.R. Tolkien"
4 )
5
6 book.category

```

'book'

The distinction is:

```
1 class attribute
2     associated with the class
3
4 instance attribute
5     associated with an individual instance
```

Class attributes are useful for information that logically belongs to the class as a whole rather than to one particular object.

4 Object Initialisation

When an object is created, it often needs an initial state.

Python classes commonly define this state using the special method:

```
1 __init__()
```

For example:

```
1 class Book:
2
3     def __init__(
4         self,
5         title,
6         author,
7         electronic=False
8     ):
9         self.title = title
10        self.author = author
11        self.electronic = electronic
```

An object can then be initialised as:

```
1 book = Book(
2     title="The Lord of the Rings",
3     author="J.R.R. Tolkien"
4 )
```

Its attributes are:

```
1 book.title
```

'The Lord of the Rings'

```
1 book.author
```

'J.R.R. Tolkien'

```
1 book.electronic
```

False

The call:

```
1 Book(...)
```

creates an instance and causes its initialisation machinery to run. In ordinary class definitions, `__init__()` is then responsible for initialising the already created instance.

For this reason, although `__init__()` is often informally called a *constructor*, it is more precisely an **initialiser**.

The distinction is normally not important when simply using classes, but it provides a more accurate mental model of Python's object system.

5 The self Parameter

The first parameter of an instance method conventionally has the name:

```
1 self
```

`self` refers to the particular instance on which the method is operating.

Consider:

```
1 class Book:
2
3     def __init__(
4         self,
5         title,
6         author
7     ):
8         self.title = title
9         self.author = author
```

When:

```
1 book = Book(
2     "The Hobbit",
3     "J.R.R. Tolkien"
4 )
```

is evaluated, the assignments:

```
1 self.title = title
2 self.author = author
```

store the supplied values on that particular instance.

Thus:

```
1 self.title
```

means:

the title attribute belonging to this instance.

If another object is created:

```
1 another_book = Book(
2     "1984",
3     "George Orwell"
4 )
```

then:

```
1 book.title
```

'The Lord of the Rings'

and:

```
1 another_book.title
```

'1984'

refer to attributes stored on two different objects.

self is therefore what allows the same method definition to operate on different instances while accessing the state of the appropriate object.

6 Default Attribute Values

Initialisation parameters can use default values in the same way as ordinary function parameters.

For example:

```
1 class Book:
2
3     def __init__(
4         self,
5         title,
6         author="",
7         electronic=False
8     ):
9         self.title = title
10        self.author = author
11        self.electronic = electronic
```

A book can now be created without explicitly specifying every attribute:

```
1 book = Book(
2     title="One Thousand and One Nights"
3 )
4
5 book.title
```

```
'One Thousand and One Nights'
```

```
1 book.author
```

```
"
```

```
1 book.electronic
```

```
False
```

Defaults are useful when a value has a meaningful and predictable default state.

They should not be used merely to avoid specifying information that is logically required for a valid object.

7 Inspecting Instance Attributes

For ordinary Python objects, instance attributes are commonly stored in an internal dictionary.

It can be inspected using:

```
1 book.__dict__
```

```
{'title': 'One Thousand and One Nights', 'author': '', 'electronic': False}
```

For example:

```
1 {  
2     "title": ...,  
3     "author": ...,  
4     "electronic": ...  
5 }
```

This is useful for understanding how simple Python objects store their state.

However, application code should normally access attributes through the object's public interface rather than relying directly on `__dict__`.

Not every Python object necessarily exposes instance state in this exact form.

8 Methods

Attributes represent object state.

Methods define behaviour associated with the object.

Suppose that a Book should provide a human-readable description.

```
1 class Book:
2
3     def __init__(
4         self,
5         title,
6         author,
7         electronic=False
8     ):
9         self.title = title
10        self.author = author
11        self.electronic = electronic
12
13    def describe(self):
14        return (
15            f"{self.title} "
16            f"by {self.author}"
17        )
```

An instance can then call the method:

```
1 book = Book(
2     title="The Hobbit",
3     author="J.R.R. Tolkien"
4 )
5
6 book.describe()
```

'The Hobbit by J.R.R. Tolkien'

The method has access to the object's state through self.

For example:

```
1 self.title
2 self.author
```

refer to attributes belonging to the instance that invoked the method.

8.1 Methods that Modify State

Methods can also modify object state.

Consider:

```
1 class Book:
2
3     def __init__(
4         self,
5         title,
6         author,
7         available=True
8     ):
9         self.title = title
10        self.author = author
11        self.available = available
12
13    def borrow(self):
14        self.available = False
15
16    def return_book(self):
17        self.available = True
```

Create an object:

```
1 book = Book(
2     title="The Hobbit",
3     author="J.R.R. Tolkien"
4 )
5
6 book.available
```

True

Borrowing the book changes its state:

```
1 book.borrow()  
2  
3 book.available
```

False

Returning it changes the state again:

```
1 book.return_book()  
2  
3 book.available
```

True

The object therefore combines:

```
1 state  
2   title  
3   author  
4   available  
5  
6 behaviour  
7   borrow()  
8   return_book()
```

This combination of related state and behaviour is one of the central motivations for object-oriented design.

9 Instance Methods

The methods considered above are **instance methods**.

An instance method:

- receives the instance as its first argument;
- conventionally names that argument `self`;
- can access instance attributes;
- can modify instance state;

- can call other instance methods.

For example:

```
1 class Rectangle:
2
3     def __init__(
4         self,
5         width,
6         height
7     ):
8         self.width = width
9         self.height = height
10
11     def area(self):
12         return (
13             self.width
14             * self.height
15         )
16
17     def perimeter(self):
18         return (
19             2
20             * (
21                 self.width
22                 + self.height
23             )
24         )
```

Create an instance:

```
1 rectangle = Rectangle(
2     width=4,
3     height=3
4 )
```

The methods operate on that particular rectangle:

```
1 rectangle.area()
```

12

```
1 rectangle.perimeter()
```

14

The same method definitions can be reused for another instance:

```
1 another_rectangle = Rectangle(  
2     width=10,  
3     height=5  
4 )  
5  
6 another_rectangle.area()
```

50

The code implementing `area()` does not need to know in advance which rectangle will use it.

The appropriate state is obtained through `self`.

10 Methods with Additional Parameters

Instance methods can receive additional parameters beyond `self`.

For example:

```
1 class BankAccount:  
2  
3     def __init__(  
4         self,  
5         balance=0  
6     ):  
7         self.balance = balance  
8  
9     def deposit(  
10        self,  
11        amount  
12    ):  
13        self.balance += amount
```

Create an account:

```
1 account = BankAccount(  
2     balance=100  
3 )
```

and deposit money:

```
1 account.deposit(50)  
2  
3 account.balance
```

150

Here:

```
1 self
```

identifies the account being modified, while:

```
1 amount
```

provides information associated with the particular method call.

11 Returning Values from Methods

Methods can return values exactly like ordinary functions.

For example:

```
1 class BankAccount:  
2  
3     def __init__(  
4         self,  
5         balance=0  
6     ):  
7         self.balance = balance  
8  
9     def deposit(  
10         self,  
11         amount
```

```
12 ):
13     self.balance += amount
14
15     def get_balance(self):
16         return self.balance
```

Then:

```
1 account = BankAccount(100)
2
3 account.deposit(50)
4
5 current_balance = account.get_balance()
6
7 current_balance
```

150

Whether a method should:

- return a value;
- modify state;
- or do both

depends on the semantics of the object and the operation being represented.

12 Classes as Abstractions

The main purpose of a class is not merely to group functions under a common name.

A useful class represents a coherent abstraction.

For example:

```
1 BankAccount
```

may reasonably contain:

```
1 state
2     balance
3
4 behaviour
5     deposit()
6     withdraw()
```

because the data and operations belong naturally to the same concept.

By contrast, creating a class only to collect unrelated utility functions often provides little benefit over ordinary functions or modules.

This leads to an important design principle:

Use a class when state and behaviour form a meaningful object abstraction.

Object-oriented programming is therefore a design tool, not a requirement that every piece of Python code be expressed as a class.

13 OOP in Data Science and Machine Learning

Object-oriented programming appears extensively in the Python machine learning ecosystem.

A typical estimator interface illustrates the pattern:

```
1 model = SomeEstimator(
2     parameter=value
3 )
4
5 model.fit(
6     X_train,
7     y_train
8 )
9
10 predictions = model.predict(
11     X_test
12 )
```

Conceptually:

```
1 class
2     SomeEstimator
3
4 instance
5     model
6
7 configuration/state
8     parameters
9     fitted quantities
10
11 behaviour
12     fit()
13     predict()
```

Before fitting, the object contains its configuration.

After fitting, the same object may also contain state learned from the training data.

This is one reason OOP is useful for machine learning systems: a model naturally combines:

- configuration;
- learned state;
- operations that depend on that state.

Similar patterns appear in:

- estimators;
- preprocessing transformers;
- pipelines;
- dataset abstractions;
- model wrappers;
- experiment components.

Understanding classes and objects therefore helps not only when writing custom Python classes, but also when reasoning about the interfaces of the libraries used throughout machine learning.

14 Functions or Classes?

Not every task should be implemented using object-oriented programming.

Suppose a calculation depends only on its inputs:

```

1 def mean_squared_error(
2     y_true,
3     y_pred
4 ):
5     return np.mean(
6         (y_true - y_pred) ** 2
7     )

```

There is no persistent state and no obvious object whose behaviour is being represented.

A function is therefore a natural abstraction.

By contrast, consider a fitted model that must preserve parameters learned from training data:

```

1 model
2 |— configuration
3 |— fitted parameters
4 |— fit()
5 |— predict()

```

Here an object is a natural representation because state must persist between operations.

A useful rule of thumb is:

Prefer a function when the operation is primarily a transformation of inputs into outputs. Consider a class when several operations belong to a coherent abstraction with persistent state.

The choice should follow the structure of the problem rather than a preference for one programming paradigm.

15 Static Methods

An instance method operates on a particular instance and therefore receives self.

Sometimes, however, an operation is logically related to a class without requiring access to either:

- a particular instance;
- the class itself.

In this situation, Python provides **static methods**.

A static method is declared using the `@staticmethod` decorator.

Consider the Rectangle class:

```
1 class Rectangle:
2
3     def __init__(
4         self,
5         width,
6         height
7     ):
8         self.width = width
9         self.height = height
10
11     def area(self):
12         return (
13             self.width
14             * self.height
15         )
16
17     @staticmethod
18     def have_same_size(
19         rectangle1,
20         rectangle2
21     ):
22         return (
23             rectangle1.width
24             == rectangle2.width
25             and
26             rectangle1.height
27             == rectangle2.height
28         )
```

Create two rectangles:

```
1 rectangle1 = Rectangle(
2     width=7,
3     height=5
4 )
```

```
5
6 rectangle2 = Rectangle(
7     width=7,
8     height=5
9 )
```

The static method can be called through the class:

```
1 Rectangle.have_same_size(
2     rectangle1,
3     rectangle2
4 )
```

True

Unlike an instance method, the static method does not receive:

```
1 self
```

automatically.

It also does not receive the class itself.

Conceptually:

```
1 instance method
2     receives self
3     operates naturally on instance state
4
5 static method
6     receives neither self nor cls
7     represents a utility logically associated with the class
```

The method could technically have been written as an ordinary function outside the class.

The reason for placing it inside `Rectangle` is organisational: comparing the dimensions of two rectangles is conceptually related to rectangles.

This leads to an important design question:

Does this operation genuinely belong to the abstraction represented by the class?

If not, an ordinary function may be clearer.

16 Class Methods

A **class method** receives the class itself rather than a particular instance.

It is declared using the `@classmethod` decorator.

By convention, its first parameter is named:

```
1 cls
```

For example:

```
1 class Rectangle:
2
3     def __init__(
4         self,
5         width,
6         height
7     ):
8         self.width = width
9         self.height = height
10
11     @classmethod
12     def square(
13         cls,
14         side
15     ):
16         return cls(
17             width=side,
18             height=side
19         )
```

The method can be called through the class:

```
1 square = Rectangle.square(5)
2
3 square.width
```

5

```
1 square.height
```

```
5
```

The important expression is:

```
1 cls(...)
```

Rather than explicitly writing:

```
1 Rectangle(...)
```

the method creates an instance of whichever class is represented by `cls`.

Class methods are therefore particularly useful as **alternative constructors**.

16.1 Alternative Constructors

Suppose rectangle dimensions are supplied as a string:

```
1 "10x5"
```

A class method can convert this external representation into a `Rectangle`.

```
1 class Rectangle:
2
3     def __init__(
4         self,
5         width,
6         height
7     ):
8         self.width = width
9         self.height = height
10
11     @classmethod
12     def from_string(
13         cls,
14         dimensions
15     ):
```

```

16     width, height = dimensions.split("x")
17
18     return cls(
19         width=float(width),
20         height=float(height)
21     )

```

The alternative constructor can then be used as:

```

1 rectangle = Rectangle.from_string(
2     "10x5"
3 )
4
5 rectangle.width

```

10.0

```

1 rectangle.height

```

5.0

The ordinary constructor:

```

1 Rectangle(...)

```

and the alternative constructor:

```

1 Rectangle.from_string(...)

```

create the same kind of object from different input representations.

This pattern appears frequently in Python libraries.

16.2 Instance, Static, and Class Methods

The three method types can now be compared directly.

Method type	First implicit argument	Typical purpose
Instance method	self	Operate on instance state
Class method	cls	Operate on the class or construct instances
Static method	None	Class-related utility independent of instance and class state

The distinction is primarily about **which context the method requires**.

If an operation requires the state of a particular object, it should normally be an instance method.

If it requires the class itself, a class method may be appropriate.

If it requires neither but logically belongs to the class abstraction, a static method may be appropriate.

17 Properties

Attributes can normally be accessed directly:

```
1 rectangle.width
```

Sometimes, however, a value should behave like an attribute while being computed dynamically.

Python provides the `@property` decorator for this purpose.

Consider:

```
1 class Rectangle:
2
3     def __init__(
4         self,
5         width,
6         height
7     ):
8         self.width = width
9         self.height = height
10
```

```

11 @property
12 def area(self):
13     return (
14         self.width
15         * self.height
16     )

```

Create a rectangle:

```

1 rectangle = Rectangle(
2     width=4,
3     height=3
4 )

```

The area is accessed as:

```

1 rectangle.area

```

12

rather than:

```

1 rectangle.area()

```

Although area is implemented through a method, it behaves externally like an attribute.

The value is calculated whenever it is accessed.

If the width changes:

```

1 rectangle.width = 10
2
3 rectangle.area

```

30

the property reflects the new state automatically.

17.1 Computed Attributes

Properties are particularly natural for quantities derived entirely from existing object state.

For example:

```
1 class Circle:
2
3     def __init__(
4         self,
5         radius
6     ):
7         self.radius = radius
8
9     @property
10    def diameter(self):
11        return 2 * self.radius
12
13    @property
14    def circumference(self):
15        return (
16            2
17            * np.pi
18            * self.radius
19        )
20
21    @property
22    def area(self):
23        return (
24            np.pi
25            * self.radius ** 2
26        )
```

Create a circle:

```
1 circle = Circle(
2     radius=3
3 )
```

The derived quantities are accessed naturally:

```
1 circle.diameter
```

6

```
1 circle.circumference
```

18.84955592153876

```
1 circle.area
```

28.274333882308138

These values do not need to be stored separately.

They are functions of:

```
1 circle.radius
```

Storing them independently could create inconsistent state.

For example, if both radius and area were stored independently, changing the radius would require remembering to update the area.

A computed property avoids that duplication.

18 Controlled Attribute Access

Properties can also control how an attribute is read or modified.

Suppose a rectangle should never have a non-positive width.

One possible implementation is:

```
1 class Rectangle:
2
3     def __init__(
4         self,
5         width,
6         height
7     ):
8         self.width = width
```

```

9     self.height = height
10
11     @property
12     def width(self):
13         return self._width
14
15     @width.setter
16     def width(
17         self,
18         value
19     ):
20         if value <= 0:
21             raise ValueError(
22                 "Width must be positive."
23             )
24
25         self._width = value

```

Now:

```

1 rectangle = Rectangle(
2     width=4,
3     height=3
4 )
5
6 rectangle.width

```

4

The assignment:

```

1 self.width = width

```

inside `__init__()` invokes the property setter.

A valid update works normally:

```

1 rectangle.width = 10
2
3 rectangle.width

```

10

An invalid update raises an exception:

```
1 try:
2     rectangle.width = -5
3 except ValueError as error:
4     print(error)
```

Width must be positive.

The external interface remains:

```
1 rectangle.width
```

even though access is now controlled internally.

19 Public and Non-Public Attributes

Some object-oriented languages enforce access modifiers such as:

```
1 public
2 private
3 protected
```

Python generally follows a different philosophy.

Access restrictions are largely communicated through **naming conventions** rather than strictly enforced by the language.

An attribute such as:

```
1 self.width
```

is treated as part of the public interface.

An attribute beginning with a single underscore:

```
1 self._width
```

conventionally means:

This attribute is intended for internal use.

For example:

```
1 class Temperature:
2
3     def __init__(
4         self,
5         celsius
6     ):
7         self._celsius = celsius
8
9     @property
10    def celsius(self):
11        return self._celsius
```

The underscore does not make `_celsius` inaccessible.

Python still allows:

```
1 temperature._celsius
```

The underscore communicates an interface convention rather than creating strict privacy.

This is consistent with a broader Python principle:

Encapsulation is primarily about designing a clear interface, not about making internal state impossible to access.

20 Encapsulation

Encapsulation refers to keeping related state and behaviour together while exposing a controlled interface to the rest of the program.

Consider a bank account:

```
1 class BankAccount:
2
3     def __init__(
4         self,
5         initial_balance=0
```

```

6 ):
7     if initial_balance < 0:
8         raise ValueError(
9             "Initial balance cannot be negative."
10        )
11
12    self._balance = initial_balance
13
14    @property
15    def balance(self):
16        return self._balance
17
18    def deposit(
19        self,
20        amount
21    ):
22        if amount <= 0:
23            raise ValueError(
24                "Deposit must be positive."
25            )
26
27        self._balance += amount
28
29    def withdraw(
30        self,
31        amount
32    ):
33        if amount <= 0:
34            raise ValueError(
35                "Withdrawal must be positive."
36            )
37
38        if amount > self._balance:
39            raise ValueError(
40                "Insufficient funds."
41            )
42
43        self._balance -= amount

```

Create an account:

```
1 account = BankAccount(  
2     initial_balance=100  
3 )
```

The balance can be inspected:

```
1 account.balance
```

100

and modified through meaningful operations:

```
1 account.deposit(50)  
2  
3 account.balance
```

150

```
1 account.withdraw(30)  
2  
3 account.balance
```

120

The class controls the rules governing valid state transitions.

The important idea is not merely that `_balance` begins with an underscore.

The important idea is that the object provides an interface:

```
1 balance  
2 deposit()  
3 withdraw()
```

through which its state can be used consistently.

21 Special Methods

Python classes can define methods with names surrounded by double underscores:

```
1 __init__()
2 __str__()
3 __repr__()
4 __len__()
5 ...
```

These are commonly called **special methods** or **dunder methods**.

They allow user-defined classes to participate in Python's standard syntax and built-in operations.

For example:

```
1 print(object)
```

can invoke:

```
1 object.__str__()
```

while:

```
1 len(object)
```

can invoke:

```
1 object.__len__()
```

Special methods are therefore part of Python's **data model**.

Rather than inventing a completely separate interface for custom objects, classes can integrate with ordinary Python syntax.

22 String Representation with `__str__()`

Without a custom string representation:

```

1 class Rectangle:
2
3     def __init__(
4         self,
5         width,
6         height
7     ):
8         self.width = width
9         self.height = height

```

printing an instance produces a default representation:

```

1 rectangle = Rectangle(
2     width=4,
3     height=3
4 )
5
6 print(rectangle)

```

<__main__.Rectangle object at 0x000002708273BE00>

A more useful representation can be defined using:

```

1 __str__()

```

For example:

```

1 class Rectangle:
2
3     def __init__(
4         self,
5         width,
6         height
7     ):
8         self.width = width
9         self.height = height
10
11     def __str__(self):
12         return (

```

```

13     f"Rectangle("
14     f"width={self.width}, "
15     f"height={self.height}"
16     f")"
17 )

```

Now:

```

1 rectangle = Rectangle(
2     width=4,
3     height=3
4 )
5
6 print(rectangle)

```

Rectangle(width=4, height=3)

produces a representation designed specifically for human-readable output.

The method:

```

1 __str__()

```

should return a string.

It should not itself print the string.

23 `__repr__()` and `__str__()`

Python also provides:

```

1 __repr__()

```

for an object's representation.

The conceptual distinction is:

```
1  __str__()
2      readable representation for users
3
4  __repr__()
5      informative representation for developers
```

For example:

```
1  class Rectangle:
2
3      def __init__(
4          self,
5          width,
6          height
7      ):
8          self.width = width
9          self.height = height
10
11     def __repr__(self):
12         return (
13             f"Rectangle("
14             f"width={self.width!r}, "
15             f"height={self.height!r}"
16             f")"
17         )
18
19     def __str__(self):
20         return (
21             f"{self.width} × "
22             f"{self.height} rectangle"
23         )
```

Create an object:

```
1  rectangle = Rectangle(
2      width=4,
3      height=3
4  )
```

Calling:

```
1 repr(rectangle)
```

'Rectangle(width=4, height=3)'

uses `__repr__()`.

Calling:

```
1 str(rectangle)
```

'4 × 3 rectangle'

uses `__str__()`.

And:

```
1 print(rectangle)
```

4 × 3 rectangle

uses the human-readable string representation.

A useful `__repr__()` should make the object's identity and important state easy to inspect during development.

24 Equality and Object Identity

Two objects can contain the same state without being the same object.

Consider:

```
1 rectangle1 = Rectangle(  
2     width=4,  
3     height=3  
4 )  
5  
6 rectangle2 = Rectangle(  
7     width=4,  
8     height=3  
9 )
```

They are separate instances:

```
1 rectangle1 is rectangle2
```

False

The operator:

```
1 is
```

tests **object identity**.

It asks whether two references point to the same object.

By contrast:

```
1 ==
```

tests equality according to the semantics defined for the class.

Without a custom equality implementation, user-defined objects generally compare according to their inherited default behaviour.

25 Defining Equality with `__eq__()`

A class can define its own equality semantics using:

```
1 __eq__()
```

For example:

```
1 class Rectangle:
2
3     def __init__(
4         self,
5         width,
6         height
7     ):
8         self.width = width
9         self.height = height
10
```

```

11  def __eq__(
12      self,
13      other
14  ):
15      if not isinstance(
16          other,
17          Rectangle
18      ):
19          return NotImplemented
20
21      return (
22          self.width == other.width
23          and
24          self.height == other.height
25      )

```

Create two separate objects:

```

1  rectangle1 = Rectangle(
2      width=4,
3      height=3
4  )
5
6  rectangle2 = Rectangle(
7      width=4,
8      height=3
9  )

```

They do not have the same identity:

```

1  rectangle1 is rectangle2

```

False

but they now have equal values according to the semantics of the class:

```

1  rectangle1 == rectangle2

```

True

This distinction is fundamental:

```
1 identity
2   Are these references to the same object?
3
4 equality
5   Should these objects be considered equivalent?
```

26 Other Special Methods

Python provides many additional special methods.

For example, a collection-like object may implement:

```
1 __len__()
```

A numerical object may implement operators such as:

```
1 __add__()
2 __sub__()
3 __mul__()
```

A container may implement:

```
1 __getitem__()
```

These methods allow custom objects to participate naturally in Python expressions.

For example, if a class implements:

```
1 __len__()
```

then:

```
1 len(obj)
```

can be used.

If it implements:

```
1 __add__()
```

then:

```
1 obj1 + obj2
```

can acquire a class-specific meaning.

The general pattern is:

```
1 Python syntax
2   ↓
3 special method
4   ↓
5 class-defined behaviour
```

This is one of the mechanisms through which Python achieves polymorphic behaviour across different types.

27 A Note on `__del__()`

Python also provides the special method:

```
1 __del__()
```

which may be called when an object is being finalised.

For example:

```
1 class Example:
2
3     def __del__(self):
4         print(
5             "Object is being finalised."
6         )
```

However, `__del__()` should not generally be understood as an ordinary destructor that must be explicitly implemented for each class.

Similarly:

```
1 del object_name
```

deletes a reference bound to that name; it does not necessarily imply that the underlying object is immediately destroyed.

Object lifetime depends on Python's memory-management mechanisms and on whether other references to the object still exist.

For this reason, `__del__()` is rarely necessary in ordinary data science or machine learning code.

Resources that require deterministic cleanup are usually better handled through mechanisms designed explicitly for that purpose, such as context managers.

The practical lesson is therefore:

Know that `__del__()` exists, but do not rely on it as the standard mechanism for managing object lifetime or external resources.

28 Designing a Small Class

The concepts considered so far can be combined into a more complete example.

Suppose that a dataset object should store:

- a name;
- a feature matrix;
- a target vector.

It should also provide information about the number of observations and features.

```
1 class Dataset:
2
3     def __init__(
4         self,
5         name,
6         X,
7         y
8     ):
9         if len(X) != len(y):
10             raise ValueError(
11                 "X and y must contain "
12                 "the same number of observations."
```

```

13     )
14
15     self.name = name
16     self.X = X
17     self.y = y
18
19     @property
20     def n_samples(self):
21         return len(self.X)
22
23     @property
24     def n_features(self):
25         return self.X.shape[1]
26
27     def __repr__(self):
28         return (
29             f"Dataset("
30             f"name={self.name!r}, "
31             f"n_samples={self.n_samples}, "
32             f"n_features={self.n_features}"
33             f")"
34     )

```

Create some data:

```

1 X = np.array([
2     [1.2, 3.4],
3     [2.1, 1.8],
4     [3.5, 2.7]
5 ])
6
7 y = np.array([
8     0,
9     1,
10    0
11 ])

```

and construct the object:

```

1 dataset = Dataset(
2     name="example",
3     X=X,
4     y=y
5 )
6
7 dataset

```

Dataset(name='example', n_samples=3, n_features=2)

Its computed properties are:

```

1 dataset.n_samples

```

3

```

1 dataset.n_features

```

2

This class combines several OOP concepts:

```

1 Dataset
2 |
3 |— instance state
4 |   |— name
5 |   |— X
6 |   |— y
7 |
8 |— validation
9 |   |— compatible sample counts
10 |
11 |— computed properties
12 |   |— n_samples
13 |   |— n_features
14 |
15 |— representation
16 |   |— __repr__()

```

The class provides a useful abstraction because the state and the operations describing that state belong naturally together.

29 OOP and Interface Design

An important consequence of encapsulation is that code using an object should ideally depend on its **public interface** rather than on unnecessary details of its internal implementation.

For the Dataset class, external code can rely on:

```
1 dataset.name
2 dataset.X
3 dataset.y
4 dataset.n_samples
5 dataset.n_features
```

without needing to know exactly how every derived value is calculated internally.

This separation becomes increasingly important as software grows.

A class can change its internal implementation while preserving the same external interface.

In machine learning libraries, this is one reason users can interact with models through stable operations such as:

```
1 fit()
2 predict()
3 transform()
```

without needing to understand the internal implementation of every estimator.

A well-designed class therefore does more than group code.

It defines a **contract** between the object and the code that uses it.

30 Inheritance

Inheritance allows a class to reuse and extend the behaviour of another class.

The existing class is commonly called the **parent class**, **base class**, or **superclass**.

The new class is commonly called the **child class**, **derived class**, or **subclass**.

Conceptually:

```
1 parent class
2   ↓
3 child class
```

The child class can inherit:

- attributes;
- methods;
- properties;
- other behaviour defined by the parent.

Inheritance is useful when several classes represent related concepts that share common structure or behaviour.

Consider two classes representing children and adults.

Without inheritance, we might write:

```
1 class Child:
2
3     def __init__(
4         self,
5         name,
6         age
7     ):
8         self.name = name
9         self.age = age
10        self.is_adult = False
11
12
13 class Adult:
14
15     def __init__(
16         self,
17         name,
18         age
19     ):
20         self.name = name
21         self.age = age
22         self.is_adult = True
```

The classes contain considerable duplicated code.

Both objects have:

```
1 name
2 age
```

and differ mainly in the value of:

```
1 is_adult
```

A more general abstraction is a Person class.

```
1 class Person:
2
3     def __init__(
4         self,
5         name,
6         age
7     ):
8         self.name = name
9         self.age = age
10
11     def introduce(self):
12         return (
13             f"My name is {self.name} "
14             f"and I am {self.age} years old."
15         )
```

The more specialised classes can inherit from Person.

```
1 class Child(Person):
2
3     is_adult = False
4
5
6 class Adult(Person):
7
8     is_adult = True
```

Create instances:

```

1 child = Child(
2     name="Alice",
3     age=12
4 )
5
6 adult = Adult(
7     name="David",
8     age=35
9 )

```

Both inherited the initialisation logic:

```

1 child.name

```

'Alice'

```

1 adult.name

```

'David'

and the method:

```

1 child.introduce()

```

'My name is Alice and I am 12 years old.'

```

1 adult.introduce()

```

'My name is David and I am 35 years old.'

The subclasses extend the common Person abstraction with more specific meaning.

31 Single Inheritance

When a class inherits from one parent class, the relationship is called **single inheritance**.

The syntax is:

```
1 class ChildClass(ParentClass):
2     ...
```

For example:

```
1 class Person:
2
3     def __init__(
4         self,
5         name,
6         age
7     ):
8         self.name = name
9         self.age = age
10
11
12 class Employee(Person):
13
14     def work(self):
15         return (
16             f"{self.name} is working."
17         )
```

Create an employee:

```
1 employee = Employee(
2     name="Alice",
3     age=30
4 )
```

The subclass has access to inherited state:

```
1 employee.name
```

'Alice'

and its own additional behaviour:

```
1 employee.work()
```

'Alice is working.'

The relationship can be described as:

```
1 Person
2   ↓
3 Employee
```

An Employee **is a** Person with additional specialised behaviour.

This “is-a” interpretation is useful when evaluating whether inheritance is conceptually appropriate.

32 Inheritance Chains

A subclass can itself become the parent of another class.

For example:

```
1 class Person:
2     ...
3
4
5 class Child(Person):
6     ...
7
8
9 class Teenager(Child):
10    ...
```

The relationship becomes:

```
1 Person
2   ↓
3 Child
4   ↓
5 Teenager
```

A Teenager inherits behaviour from both:

```
1 Child
2 Person
```

through the inheritance hierarchy.

Inheritance can therefore build progressively more specialised abstractions.

However, deep inheritance hierarchies can become difficult to reason about.

Inheritance should generally be used to represent meaningful conceptual relationships rather than simply to avoid writing a few repeated lines of code.

33 Extending a Parent Class

A subclass can add new attributes and methods while preserving inherited behaviour.

Consider:

```
1 class Person:
2
3     def __init__(
4         self,
5         name,
6         age
7     ):
8         self.name = name
9         self.age = age
10
11
12 class Employee(Person):
13
14     def __init__(
15         self,
16         name,
17         age,
18         employee_id
19     ):
20         self.name = name
21         self.age = age
22         self.employee_id = employee_id
```

This works, but the assignments:

```
1 self.name = name
2 self.age = age
```

duplicate logic already present in the parent class.

Python provides `super()` to avoid this duplication.

34 `super()`

`super()` provides access to the next implementation in the method resolution order, which in ordinary single inheritance corresponds naturally to the parent-class implementation.

The previous subclass can be written as:

```
1 class Person:
2
3     def __init__(
4         self,
5         name,
6         age
7     ):
8         self.name = name
9         self.age = age
10
11
12 class Employee(Person):
13
14     def __init__(
15         self,
16         name,
17         age,
18         employee_id
19     ):
20         super().__init__(
21             name=name,
22             age=age
23         )
24
25         self.employee_id = employee_id
```

Create an employee:

```
1 employee = Employee(  
2     name="Alice",  
3     age=30,  
4     employee_id="E001"  
5 )
```

The call:

```
1 super().__init__(  
2     name=name,  
3     age=age  
4 )
```

reuses the initialisation logic already defined by Person.

The subclass then adds only its specialised state:

```
1 self.employee_id
```

This makes the relationship between the two implementations explicit.

35 Method Overriding

A subclass can provide a new implementation of a method already defined by its parent class.

This is called **method overriding**.

Consider:

```
1 class Person:  
2  
3     def introduce(self):  
4         return "I am a person."  
5  
6  
7 class Employee(Person):  
8  
9     def introduce(self):  
10        return "I am an employee."
```

Create instances:

```
1 person = Person()  
2 employee = Employee()
```

Calling:

```
1 person.introduce()
```

'I am a person.'

uses the implementation in Person.

Calling:

```
1 employee.introduce()
```

'I am an employee.'

uses the implementation in Employee.

The subclass has overridden the inherited behaviour.

Conceptually:

```
1 Person.introduce()  
2   ↓ overridden by  
3 Employee.introduce()
```

The interface remains the same:

```
1 introduce()
```

but the behaviour depends on the object's class.

36 Extending an Overridden Method

Overriding does not always mean replacing the parent implementation completely.

A subclass can reuse the inherited behaviour and extend it.

Consider:

```

1 class Person:
2
3     def __init__(
4         self,
5         name
6     ):
7         self.name = name
8
9     def introduce(self):
10        return (
11            f"My name is {self.name}."
12        )

```

A specialised subclass might add more information:

```

1 class Employee(Person):
2
3     def __init__(
4         self,
5         name,
6         role
7     ):
8         super().__init__(name)
9
10        self.role = role
11
12    def introduce(self):
13        base_introduction = (
14            super().introduce()
15        )
16
17        return (
18            f"{base_introduction} "
19            f"I work as a {self.role}."
20        )

```

Create an employee:

```
1 employee = Employee(  
2     name="Alice",  
3     role="Data Scientist"  
4 )  
5  
6 employee.introduce()
```

'My name is Alice. I work as a Data Scientist.'

The child method extends rather than duplicates the parent method.

This is one of the main practical uses of `super()`.

37 Polymorphism

Polymorphism means that the same interface can be used with objects of different types, while the behaviour depends on the object involved.

Python already provides many familiar examples.

For instance:

```
1 len("Python")
```

6

and:

```
1 len([1, 2, 3, 4])
```

4

use the same function:

```
1 len()
```

with different object types.

The objects respond according to the behaviour defined by their respective classes.

37.1 Polymorphism through Methods

Consider:

```
1 class Person:
2
3     def introduce(self):
4         return "I am a person."
5
6
7 class Employee(Person):
8
9     def introduce(self):
10        return "I am an employee."
11
12
13 class Student(Person):
14
15     def introduce(self):
16        return "I am a student."
```

Create several objects:

```
1 objects = [
2     Person(),
3     Employee(),
4     Student()
5 ]
```

The same method call can be applied to each object:

```
1 for obj in objects:
2     print(
3         obj.introduce()
4     )
```

I am a person.

I am an employee.

I am a student.

The expression:

```
1 obj.introduce()
```

has the same syntactic form each time.

The object's class determines the behaviour.

This is polymorphism.

38 Duck Typing

Python often relies on a broader form of polymorphism sometimes described as **duck typing**.

The central idea is:

Code often cares more about whether an object supports the required behaviour than about whether it belongs to one particular class hierarchy.

Consider:

```
1 class Dog:
2
3     def speak(self):
4         return "Woof"
5
6
7 class Person:
8
9     def speak(self):
10        return "Hello"
```

The classes are unrelated by inheritance.

Nevertheless:

```
1 def make_speak(obj):
2     return obj.speak()
```

works with both:

```
1 make_speak(  
2     Dog()  
3 )
```

'Woof'

```
1 make_speak(  
2     Person()  
3 )
```

'Hello'

The function requires only that the object provide:

```
1 .speak()
```

This is an important characteristic of Python's object model.

Polymorphism does not require every object to inherit from the same explicitly declared parent class.

39 Polymorphism in Machine Learning APIs

Machine learning libraries make extensive use of common interfaces.

For example, several different estimators may support:

```
1 model.fit(  
2     X_train,  
3     y_train  
4 )  
5  
6 model.predict(  
7     X_test  
8 )
```

The internal algorithms can differ substantially:

```
1 linear regression
2 random forest
3 gradient boosting
4 neural model
5 ...
```

yet client code can interact with them through a shared conceptual interface.

This makes code such as:

```
1 for model in models:
2     model.fit(
3         X_train,
4         y_train
5     )
6
7     predictions = model.predict(
8         X_test
9     )
```

possible.

The value is not merely shorter syntax.

A stable interface allows code to depend on **behaviour** rather than on implementation details.

40 Multiple Inheritance

Python also allows a class to inherit from more than one parent class.

This is called **multiple inheritance**.

The syntax is:

```
1 class Child(
2     ParentA,
3     ParentB
4 ):
5     ...
```

For example:

```

1 class Employee:
2
3     def work(self):
4         return "Working"
5
6
7 class Calendar:
8
9     def show_schedule(self):
10        return "Showing schedule"
11
12
13 class BusinessPerson(
14     Employee,
15     Calendar
16 ):
17     pass

```

Create an instance:

```

1 person = BusinessPerson()

```

The object has access to behaviour inherited from both classes:

```

1 person.work()

```

'Working'

```

1 person.show_schedule()

```

'Showing schedule'

Conceptually:

```

1 Employee ———┐
2         |—— BusinessPerson
3 Calendar ———┘

```

Multiple inheritance can be useful when a class genuinely combines several independent behaviours.

However, it introduces additional complexity when parent classes define methods with the same name.

41 Method Resolution Order

Consider:

```
1 class A:
2
3     def identify(self):
4         return "A"
5
6
7 class B:
8
9     def identify(self):
10        return "B"
11
12
13 class C(
14     A,
15     B
16 ):
17     pass
```

Calling:

```
1 C().identify()
```

'A'

requires Python to decide which implementation should be used.

Python resolves such situations according to the **method resolution order**, or **MRO**.

The MRO can be inspected using:

```
1 C.mro()
```

```
[__main__.C, __main__.A, __main__.B, object]
```

or:

```
1 C.__mro__
```

```
(__main__.C, __main__.A, __main__.B, object)
```

For this example, the sequence begins conceptually as:

```
1 C
2 A
3 B
4 object
```

Therefore:

```
1 C().identify()
```

uses the implementation from A.

The rule should not be reduced simply to:

Python always chooses the first parent written in the class definition.

That description works for some simple cases but does not capture Python's complete inheritance algorithm.

The more precise statement is:

Python searches classes according to the method resolution order.

This becomes particularly important in more complex multiple-inheritance hierarchies.

42 super() and the MRO

The behaviour of `super()` is also defined in relation to the MRO.

Consider:

```

1 class A:
2
3     def identify(self):
4         return "A"
5
6
7 class B(A):
8
9     def identify(self):
10        return (
11            "B -> "
12            + super().identify()
13        )

```

Calling:

```

1 B().identify()

```

'B -> A'

moves from:

```

1 B

```

to the next class in the MRO:

```

1 A

```

In multiple inheritance, the same principle applies.

`super()` does not literally mean:

Call my parent.

A more accurate mental model is:

Continue method lookup from the next class in the current MRO.

For ordinary single inheritance the two interpretations usually produce the same practical result, but the second is more general and accurate.

43 Cooperative Multiple Inheritance

Multiple inheritance becomes substantially easier to manage when classes cooperate through `super()`.

Consider:

```
1 class A:
2
3     def process(self):
4         return ["A"]
5
6
7 class B(A):
8
9     def process(self):
10        result = super().process()
11        result.append("B")
12        return result
13
14
15 class C(B):
16
17     def process(self):
18        result = super().process()
19        result.append("C")
20        return result
```

Calling:

```
1 C().process()
```

`['A', 'B', 'C']`

follows the MRO and allows each implementation to contribute to the final behaviour.

This pattern is sometimes called **cooperative inheritance**.

It becomes especially relevant in frameworks that use mixins or multiple inheritance deliberately.

For introductory analytical code, however, simple inheritance structures are usually easier to maintain.

44 Inheritance from Built-In Classes

Python also allows user-defined classes to inherit from built-in types.

For example:

```
1 class PositiveInt(int):
2
3     def __new__(
4         cls,
5         value
6     ):
7         if value <= 0:
8             raise ValueError(
9                 "Value must be positive."
10            )
11
12        return super().__new__(
13            cls,
14            value
15        )
```

Create a value:

```
1 value = PositiveInt(5)
```

It behaves like an integer:

```
1 value + 10
```

15

and is also an instance of the custom class:

```
1 isinstance(
2     value,
3     PositiveInt
4 )
```

True

Subclassing built-in immutable types sometimes requires working with `__new__()` rather than only `__init__()`, because object creation and initialisation are separate stages.

This is a more advanced example, but it reinforces an earlier point: `__init__()` initialises an object; it does not perform every aspect of object creation.

45 When Inheritance Is Appropriate

Inheritance is useful when the child class represents a genuine specialisation of the parent.

For example:

```
1 Person
2   ↓
3 Employee
```

can be interpreted naturally as:

An Employee is a Person.

Similarly:

```
1 RegularPolygon
2   ↓
3 Square
```

can be interpreted as:

A Square is a RegularPolygon.

Inheritance becomes less convincing when the relationship cannot naturally be described as “is-a”.

For example:

```
1 Car
2 Engine
```

A car is not an engine.

Instead:

A car has an engine.

This type of relationship is usually better represented using **composition**.

46 Composition

Composition constructs an object from other objects rather than inheriting their behaviour.

Consider:

```
1 class Engine:
2
3     def start(self):
4         return "Engine started"
5
6
7 class Car:
8
9     def __init__(
10         self,
11         engine
12     ):
13         self.engine = engine
14
15     def start(self):
16         return self.engine.start()
```

Create the component:

```
1 engine = Engine()
```

and inject it into the composed object:

```
1 car = Car(
2     engine=engine
3 )
```

The car delegates engine-related behaviour:

```
1 car.start()
```

'Engine started'

The relationship is:

```
1 Car
2   has an
3   Engine
```

rather than:

```
1 Car
2   is an
3   Engine
```

46.1 Composition versus Inheritance

A useful distinction is:

```
1 inheritance
2   "is-a"
3
4 composition
5   "has-a"
```

Inheritance shares behaviour through a type hierarchy.

Composition assembles behaviour from collaborating objects.

Composition often provides greater flexibility because a component can be replaced without changing the outer object's inheritance hierarchy.

For example:

```
1 class ElectricEngine:
2
3     def start(self):
4         return "Electric motor started"
```

The same Car class can now use:

```
1 electric_car = Car(
2     engine=ElectricEngine()
3 )
4
5 electric_car.start()
```

'Electric motor started'

The Car class does not need to inherit from every possible engine implementation.

47 Composition in Machine Learning Systems

Composition appears naturally in machine learning workflows.

A pipeline may conceptually contain:

```
1 Pipeline
2 |
3 |— preprocessing component
4 |— feature-selection component
5 |— estimator component
```

The pipeline **has** these components.

It is not necessarily useful to model the relationship as:

```
1 Pipeline is a Scaler
2 Pipeline is a Classifier
```

Composition makes the architecture clearer.

For example, conceptually:

```
1 pipeline = Pipeline(
2     transformer=transformer,
3     estimator=model
4 )
```

The outer object coordinates several specialised objects.

This pattern is often more maintainable than constructing a deep inheritance hierarchy.

48 Prefer Composition or Inheritance?

Neither technique is universally superior.

Inheritance can be useful when:

- the subclass is genuinely a specialised form of the parent;
- the parent defines a stable interface;
- substituting the subclass where the parent is expected makes conceptual sense.

Composition can be useful when:

- an object contains or coordinates another object;
- behaviour should be replaceable independently;
- several components collaborate;
- inheritance would create artificial relationships.

A useful design question is:

Is this object a specialised version of another object, or does it contain another object as one of its components?

The answer often suggests the appropriate design.

49 Dataclasses

The classes written so far define `__init__()` manually.

For classes whose primary role is to hold structured data, Python provides the `dataclasses` module.

The `@dataclass` decorator can generate common methods automatically.

For example:

```
1 from dataclasses import dataclass
```

Define:

```

1 @dataclass
2 class Observation:
3
4     feature1: float
5     feature2: float
6     target: int

```

An object can be constructed naturally:

```

1 observation = Observation(
2     feature1=1.2,
3     feature2=3.4,
4     target=1
5 )
6
7 observation

```

Observation(feature1=1.2, feature2=3.4, target=1)

The decorator automatically creates an appropriate `__init__()`.

It also provides a useful representation:

```

1 print(observation)

```

Observation(feature1=1.2, feature2=3.4, target=1)

and value-based equality:

```

1 observation1 = Observation(
2     1.2,
3     3.4,
4     1
5 )
6
7 observation2 = Observation(
8     1.2,
9     3.4,
10    1
11 )
12
13 observation1 == observation2

```

True

Without a custom implementation, ordinary classes would not automatically provide this field-based equality behaviour.

50 Default Values in Dataclasses

Fields can have defaults:

```
1 @dataclass
2 class Experiment:
3
4     name: str
5     random_state: int = 42
6     verbose: bool = False
```

Create an object:

```
1 experiment = Experiment(
2     name="baseline"
3 )
4
5 experiment
```

Experiment(name='baseline', random_state=42, verbose=False)

The generated interface is similar to writing an ordinary class manually, but with substantially less boilerplate.

51 Dataclasses Are Still Classes

A dataclass can contain methods:

```
1 @dataclass
2 class Point2D:
3
4     x: float
5     y: float
```

```

6
7  @property
8  def norm(self):
9      return np.sqrt(
10         self.x ** 2
11         + self.y ** 2
12     )

```

Create:

```

1 point = Point2D(
2     x=3,
3     y=4
4 )
5
6 point.norm

```

`np.float64(5.0)`

The `@dataclass` decorator does not create a different programming paradigm.

It automates common class machinery for data-oriented objects.

52 When to Use a Dataclass

A dataclass is particularly useful when a class primarily represents structured state.

Examples might include:

```

1 ModelConfiguration
2 ExperimentResult
3 Observation
4 HyperparameterSet
5 TrainingMetadata

```

For example:

```

1 @dataclass
2 class ModelResult:
3
4     model_name: str
5     rmse: float
6     training_time: float

```

Then:

```

1 result = ModelResult(
2     model_name="Random Forest",
3     rmse=3.8,
4     training_time=12.4
5 )
6
7 result

```

ModelResult(model_name='Random Forest', rmse=3.8, training_time=12.4)

This is often clearer than manually writing:

```

1 __init__()
2 __repr__()
3 __eq__()

```

when those methods would simply operate mechanically on stored fields.

53 OOP as a Design Choice

The concepts introduced in this chapter should not lead to the conclusion that more classes automatically produce better Python code.

Consider three possible abstractions:

```

1 function
2 class
3 dataclass

```

Each serves a different purpose.

A **function** is often appropriate when:

```
1 inputs
2   ↓
3 transformation
4   ↓
5 output
```

with little or no persistent state.

A **class** becomes useful when:

```
1 state
2 +
3 behaviour
4 +
5 lifecycle
```

form a coherent abstraction.

A **dataclass** becomes useful when:

```
1 structured state
```

is the dominant concern and common object machinery would otherwise be repetitive.

The important skill is therefore not merely knowing how to define classes.

It is recognising when object-oriented structure improves the representation of the problem.

54 OOP in a Machine Learning Workflow

Consider a simple transformer that learns a mean and standard deviation from training data.

```
1 class Standardizer:
2
3     def __init__(self):
4         self.mean_ = None
5         self.std_ = None
6
```

```

7  def fit(
8      self,
9      X
10 ):
11     self.mean_ = np.mean(
12         X,
13         axis=0
14     )
15
16     self.std_ = np.std(
17         X,
18         axis=0
19     )
20
21     return self
22
23 def transform(
24     self,
25     X
26 ):
27     if (
28         self.mean_ is None
29         or self.std_ is None
30     ):
31         raise RuntimeError(
32             "The transformer must be fitted "
33             "before calling transform()."
34         )
35
36     return (
37         X - self.mean_
38     ) / self.std_
39
40 def fit_transform(
41     self,
42     X
43 ):
44     return (
45         self
46         .fit(X)

```

```
47     .transform(X)
48 )
```

The object has two distinct states.

Before fitting:

```
1 mean_ = None
2 std_ = None
```

After fitting:

```
1 mean_ = learned feature means
2 std_ = learned feature standard deviations
```

Create some data:

```
1 X = np.array([
2     [1.0, 10.0],
3     [2.0, 20.0],
4     [3.0, 30.0],
5     [4.0, 40.0]
6 ])
```

Fit the transformer:

```
1 standardizer = Standardizer()
2
3 standardizer.fit(X)
```

```
<__main__.Standardizer at 0x270828a97f0>
```

Inspect the learned state:

```
1 standardizer.mean_
```

```
array([ 2.5, 25. ])
```

```
1 standardizer.std_
```

```
array([ 1.11803399, 11.18033989])
```

Transform the observations:

```
1 X_standardized = (  
2     standardizer.transform(X)  
3 )  
4  
5 X_standardized
```

```
array([[ -1.34164079, -1.34164079],  
       [-0.4472136 , -0.4472136 ],  
       [ 0.4472136 ,  0.4472136 ],  
       [ 1.34164079,  1.34164079]])
```

This example demonstrates why OOP is natural for many machine learning components.

The object must preserve information learned during one operation:

```
1 fit()
```

so that the same information can be used later by:

```
1 transform()
```

A stateless function alone would need the learned quantities to be passed around separately.

The object packages:

```
1 configuration  
2 learned state  
3 behaviour
```

into one coherent abstraction.

55 Learned Attributes

Machine learning libraries often distinguish parameters supplied before fitting from quantities learned during fitting.

A common naming convention, used for example throughout estimator-style APIs, is to append an underscore to learned attributes:

```
1 mean_  
2 std_  
3 coef_  
4 classes_
```

Conceptually:

```
1 before fit()  
2     configuration exists  
3  
4 after fit()  
5     learned state also exists
```

This makes the lifecycle of a fitted object easier to reason about.

The broader OOP principle is that an object's valid behaviour can depend on its current state.

56 Interface Consistency

Suppose several transformers implement:

```
1 fit()  
2 transform()
```

and several estimators implement:

```
1 fit()  
2 predict()
```

Higher-level code can work with these objects without knowing every implementation detail.

For example:

```

1 def train_and_predict(
2     model,
3     X_train,
4     y_train,
5     X_test
6 ):
7     model.fit(
8         X_train,
9         y_train
10    )
11
12    return model.predict(
13        X_test
14    )

```

The function depends on the expected **interface**:

```

1 fit()
2 predict()

```

rather than on one exact class.

This combines several concepts considered in this chapter:

- abstraction;
- encapsulation;
- polymorphism;
- object state;
- interface design.

57 Inheritance versus Interface

A useful final distinction is that two objects do not necessarily need to share a parent class in order to expose a compatible interface.

For example:

```
1 model_a.fit(...)
2 model_a.predict(...)
3
4 model_b.fit(...)
5 model_b.predict(...)
```

may be usable through the same higher-level code even if their internal implementations are unrelated.

This is especially natural in Python because of duck typing.

Therefore:

Shared behaviour does not automatically imply that a shared inheritance hierarchy is required.

Sometimes inheritance is the correct representation.

Sometimes composition is clearer.

Sometimes independently implemented classes with a compatible interface are sufficient.

Good object-oriented design depends on choosing the simplest structure that accurately represents the relationships in the problem.

58 Problems

The following problems review the main object-oriented programming concepts introduced in this chapter. They begin with class construction and instance state, then progress through methods, properties, class and static methods, inheritance, polymorphism, composition, and object-oriented patterns used in machine learning.

58.1 Problem 1 — RationalNumber

Create a class called RationalNumber.

The constructor should receive:

```
1 numerator
2 denominator=1
```

and create the instance attributes:

```
1 numerator
2 denominator
```

Both values must be integers.

If either argument is not an integer, raise an appropriate exception.

Also prevent the denominator from being zero.

Create the following objects:

```
1 RationalNumber(3, 4)
2 RationalNumber(5)
```

Explain what state is stored independently by each instance.

58.2 Problem 2 — String Representation

Extend RationalNumber so that:

```
1 print(rational)
```

displays the object in the form:

```
1 3 / 4
```

Implement this behaviour using:

```
1 __str__()
```

Then implement:

```
1 __repr__()
```

so that:

```
1 repr(rational)
```

returns a developer-oriented representation such as:

```
1 RationalNumber(numerator=3, denominator=4)
```

Explain the conceptual difference between `__str__()` and `__repr__()`.

58.3 Problem 3 — Rational Number Behaviour

Extend `RationalNumber` with the following behaviour:

```
1 quotient
```

as a property returning:

$$\frac{\text{numerator}}{\text{denominator}}.$$

Add a method:

```
1 simplify()
```

that reduces the rational number using the greatest common divisor.

For example:

```
1 q = RationalNumber(8, 12)
2 q.simplify()
3
4 print(q)
```

should produce:

```
1 2 / 3
```

Consider whether `simplify()` should:

1. mutate the existing object; or
2. return a new `RationalNumber`.

Implement one design and justify the choice.

58.4 Problem 4 — Static Methods for Rational Arithmetic

Add static methods:

```
1 add(p, q)
2 subtract(p, q)
3 multiply(p, q)
4 divide(p, q)
```

to RationalNumber.

Each method should receive two RationalNumber instances and return a new RationalNumber.

For example:

```
1 p = RationalNumber(1, 2)
2 q = RationalNumber(2, 3)
3
4 result = RationalNumber.add(
5     p,
6     q
7 )
```

The result should represent:

$$\frac{1}{2} + \frac{2}{3} = \frac{7}{6}.$$

Explain why these methods can reasonably be static methods.

Then discuss whether implementing operator methods such as:

```
1 __add__()
```

might produce a more natural interface.

58.5 Problem 5 — Alternative Constructors

Add the following class methods to RationalNumber:

```
1 zero()
2 one()
3 from_float()
```

They should behave conceptually as:

```
1 RationalNumber.zero()
2 # 0 / 1
3
4 RationalNumber.one()
5 # 1 / 1
6
7 RationalNumber.from_float(5.4)
8 # equivalent rational representation
```

Use an appropriate standard-library mechanism to convert a finite decimal representation to a rational form.

Explain why these methods are naturally represented as class methods rather than static methods.

58.6 Problem 6 — Properties in RationalNumber

Refactor RationalNumber so that:

```
1 quotient
```

is accessed as a property:

```
1 q.quotient
```

rather than:

```
1 q.quotient()
```

Then implement a read-only property:

```
1 is_proper
```

that returns True when:

$$|\text{numerator}| < |\text{denominator}|.$$

Explain why these quantities are better represented as computed properties than stored attributes.

58.7 Problem 7 — Point3D from Point2D

Suppose the following class already exists:

```
1 class Point2D:
2
3     def __init__(
4         self,
5         x,
6         y
7     ):
8         self.x = x
9         self.y = y
10
11     def __str__(self):
12         return (
13             f"({self.x}, {self.y})"
14         )
15
16     @classmethod
17     def zero(cls):
18         return cls(
19             0,
20             0
21         )
```

Create:

```
1 Point3D
```

that inherits from Point2D.

It should add:

```
1 z
```

and appropriately override:

```
1 __init__()  
2 __str__()  
3 zero()
```

Use `super()` where appropriate.

For example:

```
1 p = Point3D(  
2     1,  
3     2,  
4     3  
5 )
```

should represent:

```
1 (1, 2, 3)
```

Explain which behaviour is inherited and which behaviour is overridden.

58.8 Problem 8 — Multiple Inheritance and Pyramid

Suppose `Square` and `Triangle` are classes representing regular polygons.

Create a class:

```
1 Pyramid
```

representing a regular square pyramid.

The object should store:

```
1 base  
2 height
```

and provide properties for:

```
1 slant_height
2 area
3 volume
```

where:

$$l = \sqrt{\left(\frac{b}{2}\right)^2 + h^2},$$

the total surface area is:

$$A = b^2 + 2bl,$$

and the volume is:

$$V = \frac{1}{3}b^2h.$$

If you use multiple inheritance, inspect:

```
1 Pyramid.mro()
```

and explain how method resolution affects calls to `super()`.

Then discuss whether composition or single inheritance might provide a simpler design.

58.9 Problem 9 — Student

Define a class:

```
1 Student
```

with instance attributes:

```
1 sid
2 name
3 gender
4 type
```

where:

```
1 type
```

defaults to:

```
1 "learning"
```

Add a method:

```
1 say_name()
```

that returns a string containing the student's name.

Create:

```
1 student1 = Student(  
2     "001",  
3     "Susan",  
4     "F"  
5 )  
6  
7 student2 = Student(  
8     "002",  
9     "Mike",  
10    "M"  
11 )
```

Call `say_name()` for both objects.

Explain why `self` does not appear explicitly in the method call.

58.10 Problem 10 — Methods with Additional Arguments

Extend `Student` with:

```
1 report(score)
```

where `score` must satisfy:

$$0 \leq \text{score} \leq 100.$$

The method should return a formatted report containing:

```
1 student name
2 student id
3 score
```

Raise an exception if the score is outside the valid range.

Explain the roles of:

```
1 self
```

and:

```
1 score
```

inside the method.

58.11 Problem 11 — Instance Independence

Create:

```
1 student1
2 student2
```

from the Student class.

Modify:

```
1 student1.type
```

without modifying:

```
1 student2.type
```

Then inspect the attributes of both instances.

Explain why modifying one instance attribute does not normally alter the corresponding attribute of another instance.

58.12 Problem 12 — Counting Instances

Add a class attribute:

```
1 n
```

to Student.

Every time a new Student object is created, increase:

```
1 Student.n
```

by one.

Then add a class method:

```
1 num_instances()
```

that returns the current number of created instances.

Explain why:

```
1 n
```

belongs more naturally to the class than to individual instances.

58.13 Problem 13 — Sensor

Define:

```
1 Sensor
```

with the instance attributes:

```
1 name
2 location
3 record_date
4 data
```

where data initially contains an empty dictionary.

Add:

```
1 add_data(t, data)
```

which stores:

```
1 "time"  
2 "data"
```

inside the dictionary.

The two supplied arrays must contain the same number of observations.

Also add:

```
1 clear_data()
```

that removes the stored measurements.

Create a sensor and add a short synthetic measurement series.

Explain why data should be created inside `__init__()` rather than as a mutable class attribute.

58.14 Problem 14 — Overriding with Accelerometer

Create:

```
1 Accelerometer
```

as a subclass of `Sensor`.

Add:

```
1 show_type()
```

returning:

```
1 I am an accelerometer.
```

Then create:

```
1 UCBAcc
```

as a subclass of `Accelerometer`.

Override:

```
1 show_type()
```

so that it additionally includes the sensor name.

Use `super()` to reuse the parent implementation.

Explain the difference between:

```
1 inheriting a method
```

and:

```
1 overriding a method
```

58.15 Problem 15 — Extending a Parent Initialiser

Create:

```
1 NewSensor
```

that inherits from `Sensor` and adds the instance attribute:

```
1 brand
```

Its constructor should call:

```
1 super().__init__(...)
```

rather than duplicating the parent initialisation logic.

Explain why the following implementation is less maintainable:

```
1 self.name = name
2 self.location = location
3 self.record_date = record_date
4 self.data = {}
```

inside the subclass.

58.16 Problem 16 — RegularPolygon

Create:

```
1 RegularPolygon
```

with:

```
1 base
2 n
```

where:

- base is the side length;
- n is the number of sides.

Implement:

```
1 __str__()
```

to produce a description of the polygon.

Create the computed properties:

```
1 apothem
2 perimeter
3 area
```

For a regular polygon:

$$a = \frac{b}{2 \tan(\pi/n)},$$

$$P = nb,$$

and:

$$A = \frac{Pa}{2}.$$

Validate that:

```
1 base > 0
2 n >= 3
```

58.17 Problem 17 — Polygon Subclasses

Create:

```
1 Triangle
2 Square
3 Pentagon
```

as subclasses of RegularPolygon.

Each subclass should only need to specify the appropriate number of sides.

For example:

```
1 square = Square(
2     base=4
3 )
```

should behave like:

```
1 RegularPolygon(
2     base=4,
3     n=4
4 )
```

Explain why this is an appropriate use of inheritance.

58.18 Problem 18 — Three-Dimensional Shapes

Create:

```
1 Tetrahedron
2 Cube
```

where:

- Tetrahedron specialises Triangle;

- Cube specialises Square.

Add appropriate:

```
1 surface_area
2 volume
```

properties.

For a cube of side a :

$$A = 6a^2,$$

$$V = a^3.$$

For a regular tetrahedron:

$$A = \sqrt{3}a^2,$$

$$V = \frac{a^3}{6\sqrt{2}}.$$

Explain which inherited properties remain meaningful and which need to be overridden.

58.19 Problem 19 — Circle

Create:

```
1 Circle
```

with:

```
1 radius
```

and properties:

```
1 diameter
2 perimeter
3 area
```

where:

$$d = 2r,$$

$$P = 2\pi r,$$

and:

$$A = \pi r^2.$$

Raise an exception if the radius is not positive.

Explain why these values should be computed rather than stored independently.

58.20 Problem 20 — Cylinder: Inheritance or Composition?

Create a class representing a cylinder with:

```
1 radius
2 height
```

and computed:

```
1 surface_area
2 volume
```

where:

$$A = 2\pi r^2 + 2\pi rh,$$

and:

$$V = \pi r^2 h.$$

First consider whether the class should inherit from Circle.

Then consider a composition-based design in which the cylinder contains a circular base object.

Implement one version and justify the design.

58.21 Problem 21 — Identity Versus Equality

Create two Rectangle objects with identical dimensions.

Before implementing `__eq__()`, compare:

```
1 rectangle1 is rectangle2
```

and:

```
1 rectangle1 == rectangle2
```

Then implement value-based equality using `__eq__()`.

Repeat the comparison.

Explain the difference between:

```
1 object identity
```

and:

```
1 object equality
```

58.22 Problem 22 — A Read-Only Property

Create:

```
1 BankAccount
```

with internal state:

```
1 _balance
```

Provide a read-only property:

```
1 balance
```

and methods:

```
1 deposit()
2 withdraw()
```

Enforce:

```
1 deposit amount > 0
2 withdrawal amount > 0
3 withdrawal <= current balance
```

Do not provide a direct setter for balance.

Explain how this design helps preserve valid object state.

58.23 Problem 23 — Class, Static, or Instance Method?

For each operation below, decide whether it should most naturally be:

```
1 instance method
2 class method
3 static method
4 ordinary function
```

and justify the decision.

1. Calculate the area of one rectangle using its stored dimensions.
2. Construct a rectangle from the string "10x5".
3. Determine whether two rectangles have identical dimensions.
4. Calculate the Euclidean distance between two arbitrary coordinate tuples.
5. Return the number of Student instances created.

There may be more than one defensible design for some items. The objective is to justify the required context.

58.24 Problem 24 — Duck Typing

Create two unrelated classes:

```
1 Dog
2 Robot
```

that both implement:

```
1 speak()
```

Then define:

```
1 def make_speak(obj):  
2     ...
```

without performing explicit type checks.

Call it with both objects.

Explain why inheritance is unnecessary for this form of polymorphism.

58.25 Problem 25 — Polymorphic Interface

Create three classes:

```
1 LinearModel  
2 TreeModel  
3 MeanModel
```

Each should implement:

```
1 fit(X, y)  
2 predict(X)
```

The internal implementations can be deliberately simple.

Store the objects in:

```
1 models
```

and execute:

```
1 for model in models:  
2     model.fit(X, y)  
3     predictions = model.predict(X)
```

Explain what makes this code polymorphic.

58.26 Problem 26 — Method Resolution Order

Consider:

```
1 class A:
2
3     def identify(self):
4         return "A"
5
6
7 class B:
8
9     def identify(self):
10        return "B"
11
12
13 class C(
14     A,
15     B
16 ):
17     pass
```

Without executing the code, predict:

```
1 C().identify()
```

Then inspect:

```
1 C.mro()
```

Reverse the parent order:

```
1 class D(
2     B,
3     A
4 ):
5     pass
```

and compare the result.

Explain why the correct general rule is based on MRO rather than simply saying “super() uses the first parent.”

58.27 Problem 27 — Cooperative super()

Create:

```
1 class A
2 class B(A)
3 class C(B)
```

where every implementation of:

```
1 process()
```

calls:

```
1 super().process()
```

and contributes one item to a list.

Verify that calling:

```
1 C().process()
```

follows the complete MRO.

Explain why calling parent classes explicitly by name can make cooperative inheritance harder to maintain.

58.28 Problem 28 — Composition

Create:

```
1 Scaler
2 Model
3 Pipeline
```

where:

```
1 Pipeline
2     has a Scaler
3     has a Model
```

The Pipeline should coordinate:

```
1 fit()
2 predict()
```

without inheriting from either component.

Explain why composition represents the relationship more naturally than multiple inheritance.

58.29 Problem 29 — Dataclass

Use:

```
1 @dataclass
```

to create:

```
1 ModelResult
```

containing:

```
1 model_name
2 rmse
3 training_time
```

Create two objects with identical values and verify value-based equality.

Then write approximately the equivalent ordinary class manually.

Identify which boilerplate the dataclass removes.

58.30 Problem 30 — Mutable Defaults in Dataclasses

Consider the conceptual class:

```
1 @dataclass
2 class Experiment:
3     metrics: list = ...
```

Explain why a mutable list should not be used directly as a shared default.

Implement the correct version using:

```
1 field(default_factory=list)
```

Create two instances and verify that modifying one instance's list does not alter the other.

58.31 Problem 31 — Dataset Abstraction

Create:

```
1 Dataset
```

storing:

```
1 name
2 X
3 y
```

Validate that:

```
1 len(X) == len(y)
```

and add properties:

```
1 n_samples
2 n_features
```

Implement:

```
1 __repr__()
```

so that the object clearly reports its structure.

Explain why this abstraction is more useful than storing X , y , and $name$ as unrelated global variables.

58.32 Problem 32 — Stateful Standardizer

Implement:

```
1 Standardizer
```

with:

```
1 fit()  
2 transform()  
3 fit_transform()
```

The object should learn:

```
1 mean_  
2 std_
```

from training data.

Before fitting:

```
1 mean_ = None  
2 std_ = None
```

After fitting, these attributes should contain one value per feature.

Raise an error if:

```
1 transform()
```

is called before:

```
1 fit()
```

Also detect features with zero standard deviation.

Explain why this object naturally requires persistent state.

58.33 Problem 33 — Separate Fit and Transform Data

Using the Standardizer from Problem 32, create:

```
1 X_train
2 X_test
```

Fit only on:

```
1 X_train
```

and transform both datasets using the same learned parameters.

Explain why fitting a new standardizer independently on `X_test` would produce an invalid evaluation workflow.

58.34 Problem 34 — Alternative Constructor for a Model Configuration

Create a dataclass:

```
1 ModelConfig
```

with fields such as:

```
1 learning_rate
2 max_depth
3 random_state
```

Add a class method:

```
1 from_dict()
```

that creates the configuration from a dictionary.

Explain why this is an alternative constructor.

58.35 Problem 35 — Inheritance or Composition?

For each relationship below, decide whether inheritance, composition, or neither is the most natural design.

1. Employee and Person
2. Car and Engine
3. Pipeline and Estimator

4. Square and RegularPolygon
5. Dataset and DataFrame
6. RandomForestModel and a generic estimator interface

Justify each decision using ideas such as:

```
1 is-a
2 has-a
3 shared interface
4 state
5 substitutability
```

58.36 Problem 36 — Refactoring a Procedural Workflow

Consider:

```
1 mean = None
2 std = None
3
4 def fit_standardizer(X):
5     global mean
6     global std
7
8     mean = X.mean(axis=0)
9     std = X.std(axis=0)
10
11
12 def transform(X):
13     return (
14         X - mean
15     ) / std
```

Refactor the workflow into a class.

Then compare the two designs in terms of:

- state ownership;
- risk of accidental interference;
- ability to maintain several independently fitted standardizers;
- testability.

58.37 Problem 37 — Two Independently Fitted Objects

Create:

```
1 standardizer_a
2 standardizer_b
```

and fit them on two different datasets.

Verify that:

```
1 standardizer_a.mean_
```

and:

```
1 standardizer_b.mean_
```

contain independent learned state.

Explain why instance state is important when several fitted models or transformers coexist in memory.

58.38 Problem 38 — Interface-Based Evaluation

Write:

```
1 def evaluate_model(
2     model,
3     X_train,
4     y_train,
5     X_test,
6     y_test
7 ):
8     ...
```

The function should assume only that model supports:

```
1 fit()
2 predict()
```

Calculate mean squared error from the predictions.

Test the function with at least two different model classes that implement the required interface.

Explain why the evaluation function does not need to know the internal implementation of either model.

58.39 Problem 39 — Diagnose a Poor Class

Consider:

```
1 class Utilities:
2
3     @staticmethod
4     def calculate_mean(values):
5         ...
6
7     @staticmethod
8     def read_csv(path):
9         ...
10
11    @staticmethod
12    def send_email(address):
13        ...
14
15    @staticmethod
16    def calculate_distance(x, y):
17        ...
```

Critique the design.

Does the class represent a coherent object abstraction?

Propose a clearer organisation using functions, modules, or more meaningful classes.

58.40 Problem 40 — Integrated OOP Reasoning

Consider:

```

1 class Transformer:
2
3     def fit(
4         self,
5         X
6     ):
7         return self
8
9     def transform(
10        self,
11        X
12    ):
13        raise NotImplementedError
14
15
16 class MeanCenterer(Transformer):
17
18     def __init__(self):
19         self.mean_ = None
20
21     def fit(
22         self,
23         X
24     ):
25         self.mean_ = X.mean(
26             axis=0
27         )
28
29         return self
30
31     def transform(
32         self,
33         X
34     ):
35         if self.mean_ is None:
36             raise RuntimeError(
37                 "Transformer is not fitted."
38             )
39
40         return X - self.mean_

```

```

41
42
43 class Pipeline:
44
45     def __init__(
46         self,
47         transformer,
48         model
49     ):
50         self.transformer = transformer
51         self.model = model
52
53     def fit(
54         self,
55         X,
56         y
57     ):
58         X_transformed = (
59             self.transformer
60             .fit(X)
61             .transform(X)
62         )
63
64         self.model.fit(
65             X_transformed,
66             y
67         )
68
69         return self
70
71     def predict(
72         self,
73         X
74     ):
75         X_transformed = (
76             self.transformer
77             .transform(X)
78         )
79
80         return self.model.predict(

```

```
81     X_transformed
82 )
```

Without executing the code, explain the complete design.

Your answer should identify:

1. the role of Transformer;
2. what behaviour MeanCenterer inherits;
3. what behaviour it overrides;
4. which state is learned during fit();
5. why fit() returns self;
6. why Pipeline uses composition rather than inheriting from its components;
7. how polymorphism allows different transformers or models to be substituted;
8. what would happen if predict() were called before the transformer had been fitted;
9. which attributes belong to the Pipeline instance and which belong to its component instances;
10. why this design resembles interfaces commonly used in machine learning libraries.

Finally, summarise the architecture using the concepts:

```
1 state
2 behaviour
3 inheritance
4 composition
5 polymorphism
6 interface
```